

FP-Devicer: Open-Source Digital Fingerprinting Middleware

Technical Whitepaper — Version 2.0.3

Gateway Corporate LLC

July 2026

1 Abstract

FP-Devicer is a lightweight, highly extensible TypeScript middleware library for server-side digital device fingerprinting. It computes a **confidence score (0–100)** between two fingerprint datasets using a **multi-component composite scorer**. The current engine blends recursive weighted comparison and TLSH similarity into a base device-similarity score, then calibrates that score with evidence-richness, field-agreement, structural-stability, entropy, missingness, and attractor-risk heuristics.

The library provides a simple `calculateConfidence` API for one-off comparisons and a full-featured `DeviceManager` paired with a pluggable `StorageAdapter` system for production-grade device identification, snapshot persistence, deduplication, adaptive weighting based on historical signal stability, and structured observability.

Version 2.0.3 extends the engine with five additional capabilities: **temporal signal decay** (confidence fades gracefully as stored snapshots age), **distribution-based drift analysis** (per-device anomaly scoring to detect abrupt fingerprint replacement), a **cross-device identity graph** (in-memory edge graph that links devices sharing IP subnets or font sets), a **pluggable attractor risk model** (operator-supplied population frequency tables replace hardcoded heuristics), and an **LSH candidate index** (MinHash Locality-Sensitive Hashing over set-valued fields for sub-linear candidate retrieval at scale).

Key design goals:

- Near-universal server compatibility (Express, Fastify, Deno/Oak, etc.)
- Extensibility via a global plugin registry (custom weights and comparators per field path)
- Strong separation on the bundled scenario benchmark suite with low-millisecond scoring latency on commodity development hardware
- Multiple storage backends: in-memory, SQLite, PostgreSQL, and Redis

The codebase is modular, well-tested, benchmarked, and documented via TypeDoc. It pairs naturally with the companion client-side collector **FP-Snatch** for a complete end-to-end fingerprinting solution.

2 Introduction & Motivation

Traditional fingerprinting libraries (e.g., FingerprintJS open-source) focus on client-side collection and hash generation. FP-Deveric shifts the intelligence to the **server side**, enabling capabilities that are impossible or impractical in the browser:

- **Persistent device IDs** that survive across browser sessions, private mode tabs, and even across different browsers on the same device
- **Adaptive scoring** that automatically down-weights signals that have proven historically volatile for a given device (e.g., screen orientation, timezone)
- **Candidate pre-filtering** to keep database queries fast even at scale — only broadly-similar fingerprints are loaded for full scoring
- **Enterprise features** such as user account linking, IP address logging, and structured metrics emission

FP-Deveric was released open-source in 2025 and has seen active development (93+ commits), with the most recent patch on March 8, 2026. It is available on GitHub at <https://github.com/gatewaycorporate/fp-deveric> and its generated TypeDoc reference documentation is hosted at <https://gatewaycorporate.github.io/fp-deveric/>.

3 Architecture Overview

FP-Deveric is structured into five logical layers, each with a clearly defined responsibility:

Data Model (`src/types/data.ts`) : Defines `FPUDataSet`, the canonical interface for all fingerprint signals collected from a browser, and the generic `FPDataSet<T>` alias. Also contains configuration types such as `ComparisonOptions` and `Comparator`.

Scoring Engine (`src/libs/confidence.ts` + `src/libs/registry.ts`) : Implements a composite confidence model built from recursive weighted comparison, TLSH fuzzy hashing, secondary agreement/stability/richness dimensions, and explicit penalties for attractor profiles and missing data. The registry is a global singleton that stores per-path weights and comparator functions contributed by built-in defaults or user plugins.

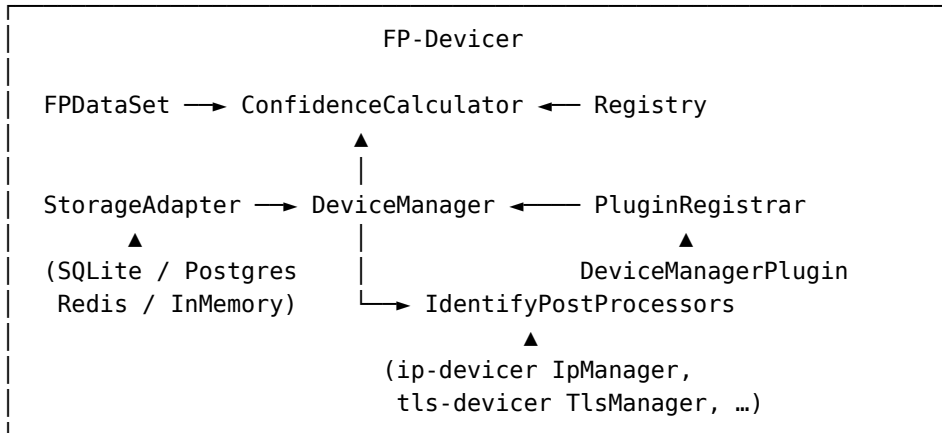
Persistence & Orchestration (`src/core/DeviceManager.ts` + `src/libs/adapters/`) : The `DeviceManager` class runs the full fingerprint matching pipeline. Storage adapters implement a common interface so that the engine code is entirely decoupled from the choice of database.

Plugin Architecture (`src/core/PluginRegistrar.ts`) : The `PluginRegistrar` manages `DeviceManagerPlugin` objects that hook into the post-identification pipeline. Each plugin implements a single `registerWith()` method that receives a `DeviceManagerLike` handle and registers one or more `IdentifyPostProcessor` callbacks. First-party companion

libraries (ip-devicer, tls-devicer) use this mechanism to enrich `IdentifyResult` with IP intelligence and TLS consistency data without coupling to FP-Devicer internals.

Observability (`src/types/observability.ts` + `src/libs/default-observability.ts`) : Defines injectable `Logger` and `Metrics` interfaces with no-op defaults, allowing callers to route telemetry to any logging or metrics platform.

The relationships among components are summarized in the class diagram below.



4 Core Components

4.1 Fingerprint Data Model

The `FPUUserDataSet` interface (defined in `src/types/data.ts`) is the canonical schema for all browser signals that FP-Snatch collects client-side and FP-Devicer consumes server-side. It contains more than 30 optional fields, allowing partial fingerprints to be processed gracefully. Relevant fields include:

Field	Type	Description
userAgent	string	Full browser user-agent string
platform	string	OS platform string (e.g., "Win32")
screen	object	Width, height, color depth, orientation
fonts	string[]	Enumerated installed system fonts
canvas	string	Hash of a rendered canvas element
webgl	string	Hash of WebGL rendering output
audio	string	Hash of an AudioContext fingerprint
plugins	object[]	Browser plugin names and descriptions
mimeType	object[]	Supported MIME types
hardwareConcurrency	number	Logical CPU core count
deviceMemory	number	Reported device RAM (GiB, rounded)
highEntropyValues	object	UA-CH high-entropy hints (brands, model, etc.)
language / languages	string / string[]	Browser locale information
timezone	string	IANA timezone identifier

All fields are optional. Missing values never cause the scorer to throw. In the current engine, recursive similarity skips absent branches, while separate missingness dimensions (`missingOneSide`, `missingBothSides`) are folded into the final composite score.

The generic `FPDataSet<T>` alias allows the scoring engine to operate on shapes other than `FPUserDataSet`, supporting advanced use cases where callers supply custom data models:

```
export type FPDataSet<T = FPUserDataSet> = T;
```

4.2 Confidence Scoring Engine

The scoring engine lives in `src/libs/confidence.ts` and is the intellectual core of `FP-Devicer`. It exposes two entry points:

- `calculateConfidence` — a pre-built default calculator, exported directly for simple use cases.
- `createConfidenceCalculator(options)` — a factory that returns a calculator instance scoped to a specific `ComparisonOptions` configuration.

The current engine should be understood as a three-stage pipeline rather than a simple two-term blend.

4.2.1 Stage 1: Recursive Device Similarity

The first stage builds `deviceSimilarity`, the strongest positive signal in the final composite. Internally, `compareRecursive` walks both fingerprint objects simultaneously using dot-notation paths.

- Primitive / leaf values delegate to the comparator registered for that path, or to exact equality when no comparator is available.
- Arrays are compared index-by-index up to the shorter length.
- Objects recurse over the union of keys found on either side.
- Missing branches contribute no recursive weight at this stage; they are handled later by explicit missingness dimensions.

The recursive structural score is:

$$S_{\text{structural}} = \frac{\sum_i w_i \cdot \text{sim}_i}{\sum_i w_i}$$

where w_i is the effective path weight and $\text{sim}_i \in [0, 1]$ is the path comparator output.

FP-Deverer then computes a holistic TLSH similarity over canonicalized JSON:

$$S_{\text{tlsh}} = \max\left(0, \frac{300 - d(h_1, h_2)}{300}\right)$$

with 300 as the fixed normalization constant used in the implementation.

The two are blended into `deviceSimilarity` with configurable `tlshWeight` (default 0.30):

$$S_{\text{device}} = 100 \cdot ((1 - w_{\text{tlsh}})S_{\text{structural}} + w_{\text{tlsh}}S_{\text{tlsh}})$$

This value is rounded to an integer in $[0, 100]$.

4.2.2 Stage 2: Secondary Dimensions

The scorer then computes seven additional dimensions, all normalized to $[0, 100]$:

- `evidenceRichness` — percentage of the default weighted field set present in each fingerprint, averaged across both sides.
- `fieldAgreement` — percentage of comparable top-level fields whose similarity meets or exceeds 0.9.
- `structuralStability` — weighted agreement across the fields the library treats as comparatively stable: `screen`, `hardwareConcurrency`, `deviceMemory`, `platform`, and `highEntropyValues`.
- `entropyContribution` — weighted agreement across the high-entropy hash fields `canvas`, `webgl`, and `audio`.

- `attractorRisk` — heuristic commonness score derived from six signals: platform family, common language prefix, mainstream browser family, common resolution, common CPU/RAM tuples, and absence of entropy-rich graphics/audio fields.
- `missingOneSide` — percentage of tracked fields present on exactly one side of the comparison.
- `missingBothSides` — percentage of tracked fields missing on both sides.

These dimensions are returned by `calculateScoreBreakdown` even when callers use `calculateConfidence` only for the final composite.

4.2.3 Stage 3: Composite Calibration and Penalties

The final score is not a raw weighted average. The implementation combines positive evidence and negative penalties separately.

Positive dimensions and fixed base weights:

- `deviceSimilarity`: 0.62
- `evidenceRichness`: 0.06
- `fieldAgreement`: 0.18
- `structuralStability`: 0.08
- `entropyContribution`: 0.06

Negative terms and fixed base weights:

- `attractor penalty`: 0.55
- `mismatch penalty`: 0.08
- `low-similarity penalty`: 0.16
- `missingOneSide`: 0.02
- `missingBothSides`: 0.01

The attractor penalty is deliberately nonlinear:

$$P_{\text{attractor}} = r^2 \cdot (0.35 + 0.4a + 0.25s)$$

where r is normalized attractor risk, a is normalized field agreement, and s is normalized device similarity.

The mismatch and low-similarity penalties suppress false positives in opposite ways:

- `computeMismatchPenalty()` grows when both agreement and holistic similarity are weak.
- `computeLowSimilarityPenalty()` activates only when `deviceSimilarity` < 60.

After positive and negative totals are combined, the implementation applies a calibration offset of +15, clamps the result to [0,100], and applies two important post-rules:

- **Non-exact ceiling** — if the canonicalized fingerprints differ, the final score is capped at `max(95, min(99, deviceSimilarity + 12))`.
- **Exact-match promotion** — if the canonicalized fingerprints are identical and `attractorRisk` < 70, the composite is forced to 100.

These rules explain why FP-Devicer's returned confidence is best read as a calibrated decision score, not a pure probability estimate.

4.2.4 Adaptive Stability Weighting

`ComparisonOptions.stabilities` lets callers provide per-field stability values. These are converted into dimension multipliers, not direct final scores. The current implementation down-weights `deviceSimilarity`, `fieldAgreement`, `structuralStability`, and `entropyContribution` when recent device history suggests those signals are volatile.

`DeviceManager` uses the same idea operationally: it derives per-field stabilities from recent snapshots and scales `DEFAULT_WEIGHTS` before scoring a candidate's latest history entry.

`DeviceManager` also computes the age of the most-recent snapshot and passes it to the scorer as `snapshotAgeMs`, enabling **temporal signal decay** (see Section 4.7). The two modulations are complementary: field-stability down-weights volatile *fields* while temporal decay down-weights old *snapshots*.

4.2.5 Options Resolution Order

When `createConfidenceCalculator()` builds its context, the current resolution order is:

1. built-in `DEFAULT_WEIGHTS`
2. global registry entries (when `useGlobalRegistry=true`)
3. `userOptions.weights` and `userOptions.comparators`
4. `defaultWeight`

For `defaultWeight` itself, the fallback order is local option, then global registry default, then 5.

One subtle but important consequence follows from the implementation: the built-in specialized comparators live in the global registry. If `useGlobalRegistry` is set to `false`, callers keep `DEFAULT_WEIGHTS` but lose those comparator specializations unless they supply replacements locally.

4.2.6 ComparisonOptions Reference

```
interface ComparisonOptions {
    weights?: Record<string, number>;
    comparators?: Record<string, Comparator>;
    stabilities?: FieldStabilityMap;
    defaultWeight?: number;           // default: 5
    tlshWeight?: number;             // default: 0.30
    maxDepth?: number;               // default: 5
    useGlobalRegistry?: boolean;     // default: true
    snapshotAgeMs?: number;          // age of candidate snapshot in ms (enables
temporal decay)
    decayHalfLifeMs?: number;        // decay half-life; default: 30 days
    attractorModel?: AttractorModel; // custom attractor risk model (see Section
```

```
4.10)
}
```

4.3 Plugin & Registry System

The registry (`src/libs/registry.ts`) is a global singleton that stores per-path comparators and weights. It is lazily seeded by `initializeDefaultRegistry()` (called from `src/libs/default-plugins.ts`) the first time any scoring function runs.

Built-in defaults include:

- Jaccard similarity for set-like arrays (`fonts`, `languages`, `plugins`, `mimeType`s)
- exact match for hash fields (`canvas`, `webgl`, `audio`)
- `screenSimilarity` for the `screen` object
- structural equality as a fallback for primitives with no registered comparator

Callers can extend or override the registry at any time using the exported helper functions:

```
registerPlugin("userAgent", {
  weight: 25,
  comparator: (a, b) =>
    levenshteinSimilarity(
      String(a ?? "").toLowerCase(),
      String(b ?? "").toLowerCase(),
    ),
});

registerWeight("canvas", 40);
registerComparator("timezone", (a, b) => (a === b ? 1 : 0.5));
setDefaultWeight(2);
unregisterWeight("audio");
unregisterComparator("audio");
```

Registry changes take effect immediately for all subsequent calls to `calculateConfidence` (including via `DeviceManager`) because the global registry is consulted at scoring time, not at calculator-creation time.

4.4 Device Management

`DeviceManager` (`src/core/DeviceManager.ts`) is the high-level orchestrator that makes FP-Deviser suitable for production use. It wraps the scoring engine with a complete fingerprint matching pipeline:

4.4.1 Pipeline Steps

1. Deduplication cache — computes the fingerprint hash of the incoming fingerprint and checks an in-memory TTL cache (implemented as a `Map`, not an `LRU`). If the same fingerprint is seen twice within the window, the cached core result is reused and the expensive persistence path is skipped.

2. Candidate pre-filtering — calls `adapter.findCandidates(incoming, minScore)` to retrieve a small set of stored device snapshots that are broadly similar to the incoming data. If an **LSH candidate index** has been built via `buildLshIndex()`, its query result is merged in to augment the adapter's structural pre-filter with set-similarity candidates (see Section 4.11).
3. Adaptive weighting & temporal decay — for each candidate, `DeviceManager` calls `adapter.getHistory(deviceId, limit=5)` and measures how stable each tracked field has been across recent history. It also computes the age of the most-recent snapshot and passes it as `snapshotAgeMs` to the scorer, enabling temporal decay (see Section 4.7).
4. Full confidence scoring — each candidate is scored against its most recent snapshot using a scorer whose field weights have been scaled by those stability estimates. The highest-scoring candidate becomes `bestMatch`.
5. Decision — if `bestMatch.confidence > matchThreshold` (default 60), its existing device ID is reused. Otherwise a new UUID-based device ID is minted.
6. Persistence — the incoming fingerprint is saved via `adapter.save()` as a new snapshot associated with the resolved device ID.
- 6a. Identity graph update — the resolved device ID is linked to any recently- active device sharing the same IPv4 /24 subnet, and to any non-matched scored candidate with $\geq 80\%$ Jaccard similarity (see Section 4.9).
7. Observability — structured log entries are emitted via the injected `Logger`. Metrics are recorded via the injected `Metrics` instance.
8. Post-processing — each registered `IdentifyPostProcessor` is called in registration order with the full pipeline outcome. Processor failures are caught individually and recorded under `enrichmentInfo.failures`.

4.4.2 Constructor Options

```
new DeviceManager(adapter, {
  matchThreshold?: number;           // default: 60
  candidateMinScore?: number;        // default: 30
  stabilityWindowSize?: number;      // default: 5
  dedupWindowMs?: number;            // default: 5000
  logger?: Logger;
  metrics?: Metrics;
});
```

4.4.3 Return Value

```
interface IdentifyResult {
  deviceId: string;
  confidence: number;
  isNewDevice: boolean;
}
```

```

matchConfidence: number;
linkedUserId?: string;
enrichmentInfo: {
  plugins: string[];
  details: Record<string, Record<string, unknown>>;
  failures: Array<{ plugin: string; message: string }>;
};
}

```

4.5 Universal Plugin Architecture for Device Managers

FP-Dever provides a second, distinct extension mechanism that operates at the pipeline level rather than the field level. Where the scoring registry customizes how individual fingerprint fields are compared, the plugin architecture customizes what happens after a device has been identified.

4.5.1 Design Overview

The architecture is built from three collaborating types:

- `DeviceManagerPlugin` — the interface all plugins implement
- `PluginRegistrar` — validates and tracks active plugins
- `DeviceManagerLike` — the minimal structural interface exposed to plugins

4.5.2 DeviceManagerPlugin Interface

```

interface DeviceManagerPlugin {
  registerWith(deviceManager: DeviceManagerLike): (() => void) | void;
}

```

Plugins are registered by calling `DeviceManager.use(plugin)`. The returned function unregisters the plugin and calls any teardown returned by `registerWith()`.

4.5.3 DeviceManagerLike Interface

```

interface DeviceManagerLike {
  registerIdentifyPostProcessor(
    name: string,
    processor: IdentifyPostProcessor,
  ): () => void;
}

```

4.5.4 IdentifyPostProcessor Lifecycle

After the core pipeline completes, `DeviceManager` invokes each registered post-processor in registration order. Each receives an `IdentifyPostProcessorPayload` and may return an `IdentifyPostProcessorResult`.

4.5.5 IdentifyEnrichmentInfo

Every `IdentifyResult` carries an `enrichmentInfo` object that accumulates the contributions of all post-processors for that request.

4.5.6 Error Isolation

If a processor throws or rejects, the error is caught, a { plugin, message } entry is appended to enrichmentInfo.failures, and execution continues with the next processor.

4.5.7 Companion Plugin: ip-devicer IpManager

IpManager from the **ip-devicer** companion package implements DeviceManagerPlugin. When registered, it hooks into the post-processor pipeline to perform real-time IP intelligence enrichment on every identify() call.

Registration:

```
import { IpManager } from "ip-devicer";
import { createSqliteAdapter, DeviceManager } from "devicer.js";

const ipManager = new IpManager({/* MaxMind DB paths, thresholds */});
const manager = new DeviceManager(createSqliteAdapter("./fp.db"), {
  matchThreshold: 60,
});

manager.use(ipManager);
```

Enriched fields added to IdentifyResult:

Field	Type	Description
ipEnrichment	object	Country, ASN, agent info, risk score, consistency score, impossible-travel flag, and proxy/VPN/Tor/hosting flags
ipRiskDelta	number	Change in risk score since the device's previous visit

The processor reads the resolved IP from context.ip (or context.resolvedIp as set by createIpMiddleware). If no IP is present in the context, the processor returns without enriching the result.

4.5.8 Companion Plugin: tls-devicer TlsManager

TlsManager from the **tls-devicer** companion package implements DeviceManagerPlugin. It cross-references the TLS/JA4 fingerprint attached to the request against stored TLS profiles for the matched device, producing a consistency score and optionally boosting the confidence value.

Registration:

```
import { createTlsMiddleware, TlsManager } from "tls-devicer";
import { createSqliteAdapter, DeviceManager } from "devicer.js";

const tlsManager = new TlsManager({/* options */});
const manager = new DeviceManager(createSqliteAdapter("./fp.db"), {
  matchThreshold: 60,
```

```
});
```

```
manager.use(tlsManager);
```

The `createTlsMiddleware()` helper extracts JA4/JA3 hashes, HTTP/2 SETTINGS, and header-order signals from each request and attaches them as `req.tlsProfile`. Pass this into `identify()` via the context:

```
app.post("/identify", async (req, res) => {  
  const result = await manager.identify(req.body, {  
    tlsProfile: req.tlsProfile,  
  });  
  res.json(result);  
});
```

Enriched fields added to `IdentifyResult`:

Field	Type	Description
<code>tlsConsistency</code>	object	Consistency score (0–1), JA4/JA3 match flags, and per-factor breakdown
<code>tlsConfidenceBoost</code>	number	Signed delta applied to <code>result.confidence</code> based on TLS consistency
<code>confidence</code>	number	Overwritten with the TLS-boosted value (original preserved in <code>matchConfidence</code>)

4.6 Storage Adapters

All adapters implement the `StorageAdapter` interface defined in `src/types/storage.ts`. The interface methods are:

Method	Description
<code>init()</code>	Create tables / open connections. Must be called before use.
<code>save(snapshot)</code>	Persist a new fingerprint snapshot.
<code>findCandidates(fp, minConfidence, limit)</code>	Return up to <code>limit</code> broadly-similar device candidates above a minimum confidence.
<code>getHistory(id, limit)</code>	Return recent snapshots for a given device ID.
<code>getAllFingerprints()</code>	Return all stored snapshots (e.g., for bulk analysis).
<code>linkToUser(id, uid)</code>	Associate a device ID with an application user ID.
<code>deleteOldSnapshots()</code>	Prune snapshots older than a configured retention window.
<code>close()</code>	(Optional) Gracefully close database connections.

The four provided adapter implementations handle the specifics:

In-memory (`createInMemoryAdapter()`) : Stores snapshots in a Map. Suitable for development, testing, and short-lived server processes. No persistence across restarts.

SQLite (`createSqliteAdapter(path)`) : Uses `better-sqlite3` (or the Deno-compatible equivalent). Creates a `fingerprints` table on first `init()`. The candidate pre-filter runs a `WHERE` clause that compares stored platform and screen values before loading the full rows. Ideal for single-process servers without external dependencies.

PostgreSQL (`createPostgresAdapter(connectionString)`) : Uses `Drizzle ORM` (`drizzle-orm/postgres-js`). Suitable for multi-process or horizontally-scaled deployments. The candidate pre-filter leverages PostgreSQL JSON operators for efficient indexed querying.

Redis (`createRedisAdapter(url)`) : Stores snapshots as JSON-serialized Redis hashes. Suitable for deployments that already operate a Redis cluster or that require very fast writes-per-second with eventual persistence.

The `AdapterFactory` class provides a configuration-driven factory for selecting and instantiating adapters without importing each adapter module directly:

```
const adapter = AdapterFactory.create("sqlite", {
  sqlite: { filePath: "./fingerprints.db" },
});
```

4.7 Temporal Signal Decay

Every stored snapshot has a `timestamp`. The further in the past a snapshot was recorded, the less confidently it should anchor a match. Temporal signal decay models this by attenuating the positive-contribution weights of the scoring engine as a function of snapshot age.

4.7.1 Decay Factor

`computeTemporalDecayFactor(snapshotAgeMs, halfLifeMs?)` computes an exponential decay factor:

$$\lambda = e^{-t / t_{1/2}}$$

where t is `snapshotAgeMs` and $t_{1/2}$ is `decayHalfLifeMs` (default: `DEFAULT_DECAY_HALF_LIFE_MS` = 30 days). The factor is in $(0, 1]$, approaching 1 for fresh snapshots and 0 for very old ones.

4.7.2 Effect on Scoring

The factor is wired into `calculateScoreBreakdown` via `ComparisonOptions.snapshotAgeMs`:

- The `fieldAgreement` positive weight is scaled by $(0.5 + 0.5 \times \lambda)$, preserving at least half its contribution even for stale snapshots.
- The `mismatch` and `lowSimilarity` negative weights are scaled by λ , fully relaxing those penalties for very old snapshots where some divergence is expected.
- The non-exact ceiling interpolates toward a floor of 70 for old snapshots, preventing stale-match inflation.

`DeviceManager` computes `snapshotAgeMs = Date.now() - history[0].timestamp` automatically and passes it into the per-candidate scorer on every identification call.

4.7.3 API

```
import { computeTemporalDecayFactor, DEFAULT_DECAY_HALF_LIFE_MS } from
"devicer.js";

// Manual factor computation
const factor = computeTemporalDecayFactor(ageMs, DEFAULT_DECAY_HALF_LIFE_MS);

// Override via ComparisonOptions
const calculator = createConfidenceCalculator({
  snapshotAgeMs: 45 * 24 * 60 * 60 * 1000, // 45 days
  decayHalfLifeMs: 14 * 24 * 60 * 60 * 1000, // 14-day half-life
});
```

4.8 Distribution-Based Drift Analysis

Normal device evolution (browser updates, font installations) produces gradual, coherent changes across individual fields. Abrupt whole-fingerprint replacement — the hallmark of canonical injection attacks — produces sudden, broad divergence across

many fields simultaneously. Drift analysis detects this pattern by comparing an incoming fingerprint against the full stored history of a verified device.

4.8.1 Algorithm

`computeDeviceDrift(incoming, history, options?)` is a pure function that:

1. Sorts history newest-first.
2. Computes `fieldStability[field]` — the average pairwise similarity between consecutive snapshots in the history window.
3. Computes `fieldDeviation[field]` — the average dissimilarity between the incoming fingerprint and every historical snapshot.
4. Derives a z-score analog per field: $z = \text{deviation} / (1 - \text{stability} + 0.05)$, capped at 5.0.
5. Aggregates into `driftScore` $\in [0, 100]$ via a weighted sum normalized by the maximum possible z-contribution.
6. Classifies the result into one of four `DriftPatternFlag` values.

4.8.2 Pattern Flags

Flag	Condition
NORMAL_AGING	$\text{driftScore} < 30$
INCREMENTAL_DRIFT	$30 \leq \text{driftScore} < 55$
ABRUPT_CHANGE	$55 \leq \text{driftScore} < 75$, or $\text{driftScore} \geq 55$ with ≤ 3 suspicious fields
CANONICAL_INJECTION	$\text{driftScore} \geq 75$ and $\text{attractorRisk} \geq 50$

4.8.3 API

```
import { computeDeviceDrift } from "deviser.js";

// Pure-function usage
const report = computeDeviceDrift(incomingFP, historicalSnapshots);
// report: { driftScore, patternFlag, suspiciousFields, fieldStability,
// fieldDeviation }

// Via DeviceManager (loads history automatically)
const report = await manager.analyzeDeviceDrift(deviceId, incomingFP);
if (report?.patternFlag === "CANONICAL_INJECTION") {
  // flag for manual review or block the request
}
```

4.9 Cross-Device Identity Graph

Multiple distinct browser profiles can originate from the same physical person or automated system. The identity graph tracks probabilistic relationships between device IDs,

enabling downstream applications to detect account-sharing, multi-browser users, and coordinated bot networks.

4.9.1 Data Model

IdentityGraph maintains a symmetric edge set keyed by a canonical pair " $\{\min(a,b)\} || \{\max(a,b)\}$ ". Each IdentityEdge carries:

- `weight` $\in (0, 0.97]$ — blend score reinforced on each observation: `weight = min(0.97, existing + newWeight × 0.5)`
- `reasons` — deduplicated list of signal types that contributed the edge
- `firstSeen`, `lastSeen`, `occurrences` — for freshness-based pruning

4.9.2 Automatic Edge Detection in DeviceManager

DeviceManager builds edges automatically during `identify()` from two signal types:

1. **Shared IP subnet** — two devices seen from the same IPv4 /24 block are linked with initial weight 0.4. The per-subnet device list is bounded to 200 entries.
2. **Font overlap** — among non-matched scored candidates, any device whose font set has Jaccard similarity ≥ 0.80 with the incoming fingerprint is linked with weight `jaccard × 0.3`.

4.9.3 API

```
// Related device lookup
const related = manager.findRelatedDevices("dev_abc");
// → [{ deviceId, weight, reasons, firstSeen, lastSeen, occurrences }, ...]

// Raw graph access
const graph = manager.getIdentityGraph();
graph.prune(30 * 24 * 60 * 60 * 1000); // evict edges older than 30 days
const edge = graph.getEdge("dev_abc", "dev_xyz");

// Utility functions
import { jaccardSimilarity, subnetKey } from "devicer.js";
```

4.10 Learned Attractor Model

The built-in `computeAttractorRisk` heuristic scores how common a fingerprint profile is relative to a set of hardcoded popular configurations. In production, the most common profiles differ by deployment context. The pluggable `AttractorModel` interface allows operators to substitute a model calibrated to their actual traffic.

4.10.1 Interface

```
export interface AttractorModel {
  score(fp: FPDataSet): number; // returns an integer in [0, 100]
}
```

Any object implementing this interface can be passed as `ComparisonOptions.attractorModel`. `calculateScoreBreakdown` calls `model.score(fp)` in place of `computeAttractorRisk`; if no model is provided, the built-in heuristic is used.

4.10.2 Built-In Implementations

`DefaultAttractorModel` wraps `computeAttractorRisk` exactly, useful when mixing default and custom models in the same application.

`FrequencyTableAttractorModel` (factory: `createFrequencyTableAttractorModel`) accepts a `FrequencyTable` that maps platform names, resolution strings, language tags, browser keyword substrings, and hardware profiles to their observed frequency `[0, 1]` in the operator's traffic. Each signal dimension is scored against the table; dimensions absent from the table fall back to the built-in heuristic for that signal. The aggregate is a weighted average of per-dimension frequencies, normalized to `[0, 100]`.

4.10.3 API

```
import { createFrequencyTableAttractorModel, createConfidenceCalculator } from
"devicer.js";
```

```
const model = createFrequencyTableAttractorModel({
  platforms:      { "Win32": 0.72, "MacIntel": 0.18 },
  resolutions:    { "1920x1080": 0.48, "2560x1440": 0.22 },
  languages:      { "en-US": 0.68 },
  browserFamilies: { "chrome": 0.60, "safari": 0.22 },
  hardwareProfiles: [{ concurrency: 8, memory: 8, frequency: 0.45 }],
});
```

```
const calculator = createConfidenceCalculator({ attractorModel: model });
const score = calculator.calculateConfidence(fp1, fp2);
```

4.11 LSH Candidate Index

The storage adapters' `findCandidates()` implementations pre-filter by structural signals (screen dimensions, platform, hardware concurrency). This misses cases where two fingerprints agree on set-valued fields (fonts, plugins, mimeTypes, languages) but differ on structural ones. The LSH candidate index fills this gap using Locality-Sensitive Hashing over token sets.

4.11.1 Algorithm

`LshIndex` uses `MinHash` with configurable `numHashes` (default 128) and `numBands` (default 16, yielding 8 rows/band). The Jaccard similarity threshold at which two fingerprints become near-certain band-bucket collisions is approximately:

$$t \approx \left(\frac{1}{B}\right)^{1/r}$$

where B is the number of bands and r is rows-per-band. With defaults ($B = 16, r = 8$): $t \approx 0.50$.

Token extraction prefixes tokens by field (f:Arial, p:PDF Reader, mt:application/pdf, l:en-US) so that tokens from different fields never collide in the shared MinHash universe.

4.11.2 DeviceManager Integration

When an LshIndex is present, `identify()` merges the LSH candidate set with the adapter's own pre-filter output. All unique device IDs are then submitted to the full confidence-scoring pass. The index is **not** updated on every `identify()` call; it is rebuilt on demand via `DeviceManager.buildLshIndex()`.

4.11.3 API

```
import { createLshIndex, buildLshIndex } from "devicer.js";

// Build index from adapter state (one-time or periodic rebuild)
await manager.buildLshIndex({ numHashes: 128, numBands: 16 });
console.log(manager.getLshIndexSize()); // number of indexed devices

// Standalone usage
const index = buildLshIndex(entries); // entries: { deviceId, fingerprint }[]
const candidates = index.query(incomingFP); // string[] of candidate device IDs
index.add("dev_new", newFP);
index.remove("dev_old");
index.clear();
```

4.11.4 Benchmark Characteristics

With default parameters (128 hashes, 16 bands) and 500 indexed fingerprints, the benchmark runner reports:

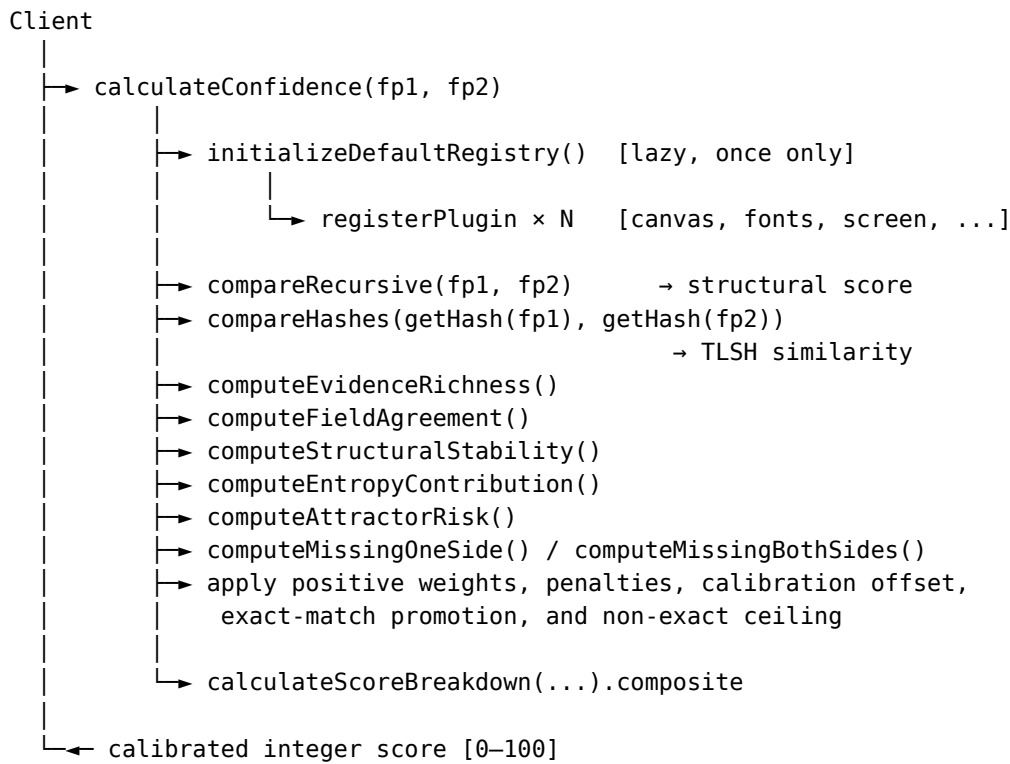
Operation	Throughput
LshIndex.query	~187,000 ops/s
LshIndex.add (single)	~183,000 ops/s
buildLshIndex (500 fps)	~95 ops/s

`query` and `add` are fast enough to be entirely in the noise of a single `identify()` call. `buildLshIndex` is intended as an offline or periodic operation, not a per-request one.

5 Code Paths & Sequence Diagrams

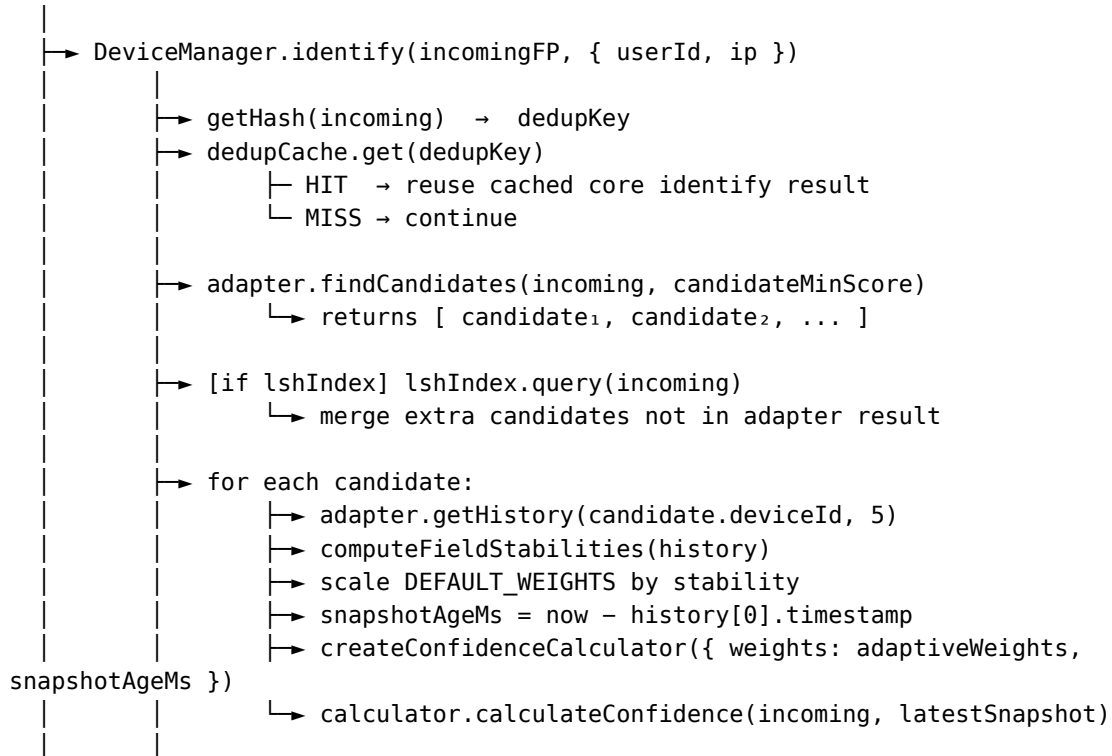
5.1 Simple Confidence Calculation

The simplest use case requires no instance management:



5.2 Full Device Identification

HTTP Request



```

    |
    |   ↳ select bestMatch (highest score)
    |   ↳ if bestMatch.score > matchThreshold:
    |       deviceId = bestMatch.deviceId (returning device)
    |   else:
    |       deviceId = crypto.randomUUID() (new device)
    |   ↳ adapter.save({ deviceId, fingerprint: incoming,
    |                   signalsHash, matchConfidence, ... })
    |   ↳ dedupCache.set(dedupKey, baseResult, dedupWindowMs)
    |   ↳ identityGraph.addEdge(subnet signal, font-overlap signal)
    |   ↳ for each identifyPostProcessor [plugin]:
    |       ↳ processor({ incoming, context, result, baseResult,
    |                   cacheHit, candidatesCount, matched,
durationMs })
    |       ↳ returns { result?, enrichmentInfo?, logMeta? }
    |       ↳ merge returned result fields → enrichedResult
    |       ↳ append enrichedResult.enrichmentInfo.details[plugin]
    |       ↳ on throw: append enrichmentInfo.failures[plugin], continue
    |   ↳ logger.info({ ...perPluginLogMeta })
    |   ↳ metrics.incrementCounter(...)
    |
    |   ↳ IdentifyResult { deviceId, confidence, isNewDevice, enrichmentInfo, ... }

```

6 Extensibility & Customization

6.1 Custom Comparators

A Comparator is any function with the signature:

```
type Comparator = (value1: any, value2: any, path?: string) => number;
```

It must return a value in $[0, 1]$ where 1.0 means identical and 0.0 means completely dissimilar. The optional path argument allows a single comparator to behave differently depending on which field it is invoked for.

Common comparator patterns include:

- **Levenshtein similarity** for string fields that change incrementally (e.g., user-agent minor version bumps).
- **Jaccard similarity** for set-valued fields (arrays where order does not matter).
- **Numeric proximity** for quantitative fields (e.g., deviceMemory treated as similar if within one doubling step).
- **Exact match** for high-entropy hash fields where any difference is significant.

6.2 Custom Storage Adapters

To integrate FP-Devider with a database not covered by the built-in adapters, implement the `StorageAdapter` interface:

```
import { FPDataSet, StorageAdapter, StoredFingerprint } from "devicer.js";

const myAdapter: StorageAdapter = {
  async init() { /* open connection, create schema */ },
  async save(snapshot: StoredFingerprint) { /* insert row */ },
  async findCandidates(fp: FPDataSet, minScore: number, limit: number) {
    /* return DeviceMatch[] of broadly-similar candidates */
    return [];
  },
  async getHistory(deviceId: string, limit: number) {
    return [];
  },
  async getAllFingerprints() {
    return [];
  },
  async linkToUser(deviceId: string, userId: string) {},
  async deleteOldSnapshots(olderThanMs: number) {},
  async close() { /* close connection */ },
};
```

6.3 Custom Observability

Inject a `Logger` and/or `Metrics` implementation into `DeviceManager` to route telemetry to your platform (e.g., Winston, Pino, StatsD, Prometheus):

```
import { createInMemoryAdapter, DeviceManager } from "devicer.js";

const manager = new DeviceManager(createInMemoryAdapter(), {
  logger: {
    info: (msg, meta) => pinoLogger.info(meta, msg),
    warn: (msg, meta) => pinoLogger.warn(meta, msg),
    error: (msg, meta) => pinoLogger.error(meta, msg),
  },
  metrics: {
    incrementCounter: (name, value = 1) => statsd.increment(name, value),
    recordGauge: (name, value) => statsd.gauge(name, value),
    recordHistogram: (name, value) => statsd.histogram(name, value),
    recordIdentify: (durationMs, confidence, isNewDevice, candidatesCount,
matched) => {
      statsd.histogram("identify_latency_ms", durationMs);
      statsd.gauge("confidence", confidence);
      if (isNewDevice) statsd.increment("new_devices");
      if (matched) statsd.increment("matches_total");
      statsd.gauge("candidates_per_identify", candidatesCount);
    },
  },
});
```

```
    },
  });
};
```

6.4 Device Manager Plugins

To integrate custom post-identification logic with any DeviceManager, implement the DeviceManagerPlugin interface exported from devicer.js:

```
import type {
  DeviceManagerLike,
  DeviceManagerPlugin,
  IdentifyPostProcessorPayload,
} from "devicer.js";

const riskPlugin: DeviceManagerPlugin = {
  registerWith(deviceManager: DeviceManagerLike) {
    const unregister = deviceManager.registerIdentifyPostProcessor(
      "risk-scorer",
      async ({ result, context }: IdentifyPostProcessorPayload) => {
        const riskScore = await myRiskApi.score(result.deviceId);
        return {
          result: { riskScore },
          enrichmentInfo: { riskScore, deviceId: result.deviceId },
          logMeta: { riskScore },
        };
      },
    );
    // Return the unregister closure as the plugin teardown function
    return unregister;
  },
};

const unregisterRisk = manager.use(riskPlugin);

// To remove the plugin cleanly:
unregisterRisk();
```

Any fields returned in result are merged onto IdentifyResult and immediately visible to processors registered after this one. The teardown function (the unregister closure from registerIdentifyPostProcessor) is called automatically when unregisterRisk() is invoked, so no manual cleanup is required.

7 Performance & Accuracy

7.1 Re-evaluated Benchmark Methodology

FP-Devicer currently ships with three benchmark entry points under src/benchmarks/:

- accuracy.bench.ts evaluates thresholded decision quality over a fixed scenario corpus.

- `scenarios.bench.ts` emits a single-seed explainability table for the canonical scenario matrix.
- `performance.bench.ts` reports coarse average latency for the scorer and for `DeviceManager.identify()` across three storage configurations.

All three run under Vitest's benchmark runner, and the raw harness output is written to `src/benchmarks/bench-results.json`.

7.1.1 Synthetic Input Families

The repository now uses two distinct synthetic-data strategies.

1. General corpus generation via `generateDataset()`. This path creates realistic base fingerprints from seeded pools covering platform, browser, locale, timezone, screen, hardware, fonts, plugins, and high-entropy browser hints. It then applies `mutate()` at severities `none`, `low`, `medium`, `high`, and `extreme`.
2. Deterministic scenario-pair generation via dedicated builders such as `generateBrowserDrift()`, `generateEnvironmentChange()`, `generatePrivacyResistance()`, `generateAdversarialPerturbation()`, `generateTravelNetworkChange()`, and `generateCommodityCollision()`.

This distinction matters. Base-profile construction is seed-driven and stable, but `generateDataset()` intentionally injects session jitter with `Math.random()` and, in some mutation paths, `Date.now()`. By contrast, the scenario-pair builders used by the bundled accuracy and scenario benchmarks are deterministic for a given numeric seed.

7.1.2 What the Scenario Corpus Actually Covers

The scenario matrix contains 17 labeled cases:

- browser drift: `minor`, `major`, `cross-browser`
- environment change: `home-office`, `external-dock`, `mobile-desktop`
- privacy resistance: `tor`, `resistant-browser`, `canvas-defender`
- adversarial perturbation: `canvas-noise`, `font-randomization`, `ua-rotation`
- travel/network change: `timezone-travel`, `vpn-activation`
- commodity collision: `corporate-fleet`, `iphone-defaults`, `public-terminal`

Per seed, 12 scenarios are labeled `same-device` and 7 are labeled `different-device`. `accuracy.bench.ts` evaluates 50 seeds, so the current benchmark corpus is exactly:

- 950 scored pairs total
- 600 genuine pairs
- 350 impostor pairs

This is a scenario-driven regression corpus, not a general population simulator.

7.1.3 Accuracy Metrics and Their Limits

`calculateMetrics()` sweeps thresholds from 0 to 100 in increments of 5 and computes precision, recall, F1, FAR, FRR, eer, and attr.

The implementation of eer should be stated precisely. It is not an interpolated equal-error-rate estimate. It is the absolute gap $|FAR - FRR|$ at each threshold, used as a simple closeness proxy. Likewise, attr is the false-accept rate computed only on attractor-zone impostor pairs.

The benchmark also acts as a regression gate: it throws if the best-F1 threshold still yields $eer > 0.08$.

7.1.4 Scenario Explainability Output

`scenarios.bench.ts` evaluates a single deterministic seed and writes a compact table containing:

- composite score
- deviceSimilarity
- fieldAgreement
- attractorRisk
- missingOneSide

This table is useful for explaining borderline scenarios that an aggregate F1 score can hide.

7.1.5 Performance Methodology

`performance.bench.ts` measures four configurations:

- `calculateConfidence`
- `DeviceManager.identify` with the in-memory adapter
- `DeviceManager.identify` with SQLite :memory:
- `DeviceManager.identify` with file-backed SQLite

Two timing mechanisms are present:

- the Vitest `bench()` harness for sustained execution
- manual batch timing used to write `performance.bench.out`

The formatted output currently reports only average milliseconds per call, derived from 1000 scorer calls and 50 `identify()` calls. Each `DeviceManager` instance is warmed with five prior identifies.

That warmup choice has an important consequence: the timed `identify()` loop reuses the same fingerprint repeatedly, so most measured requests are hot-path dedup-cache hits. The reported numbers therefore reflect warmed steady-state latency, not cold-start matching against a large candidate set.

7.2 Current Benchmark Outputs

The following values are from benchmark artifacts generated from the current repository state on July 13, 2026.

7.2.1 Accuracy

`accuracy.bench.out` currently reports the best F1 operating point at threshold 75:

Metric	Value
Best threshold	75
Precision	0.928
Recall	0.960
F1	0.943
FAR	0.129
FRR	0.040
eer proxy	0.089
Attractor impostor FAR (attr)	0.180

These results support a strong claim for the bundled scenario corpus, but not a broader claim than that. The benchmark is excellent as a regression suite and a threshold-tuning aid; it is not a substitute for field validation against live traffic.

The per-scenario summary is particularly revealing. In the latest artifact, the hardest negative classes are:

- `privacy-resistance:tor` with average score 77.30
- `commodity-collision:iphone-defaults` with average score 67.52
- `privacy-resistance:resistant-browser` with average score 67.00
- `commodity-collision:public-terminal` with average score 67.00
- `commodity-collision:corporate-fleet` with average score 66.48

The hardest positive classes are:

- `antifingerprint:metadock-max` with average score 73.96
- `privacy-resistance:canvas-defender` with average score 89.92
- `adversarial-perturbation:font-randomization` with average score 91.04

This matches the implementation's design intent: FP-Devicer is permissive with benign drift but deliberately conservative around privacy-resistant and collision-prone fingerprints.

7.2.2 Scenario Snapshot

The single-seed `scenarios.bench.out` file shows how the score breakdown behaves for individual canonical examples. A few representative rows from the current artifact are:

- browser-drift:minor -> 99
- browser-drift:cross-browser -> 54
- privacy-resistance:tor -> 80
- commodity-collision:corporate-fleet -> 70
- travel-network-change:timezone-travel -> 98
- environment-change:mobile-desktop -> 47

These values are useful sanity checks because they expose borderline cases that aggregate metrics can obscure.

7.2.3 Performance

performance.bench.out in the repository currently contains an older run artifact, while the latest local `npm run bench` execution failed to execute `performance.bench.ts` because `better-sqlite3` native bindings were not found. The retained artifact currently reports:

Configuration	Mean ms per call
calculateConfidence	1.21
DeviceManager.identify in-memory	1.47
DeviceManager.identify SQLite mem	1.44
DeviceManager.identify SQLite file	1.29

The near-parity between raw scoring and `DeviceManager.identify()` should not be read as evidence that persistence is effectively free. It is primarily a consequence of the warmed dedup-cache path exercised by the benchmark loop.

To reproduce the benchmark suite locally:

```
npm run bench
```

Artifacts are written to:

- `src/benchmarks/bench-results.json`
- `src/benchmarks/accuracy.bench.out`
- `src/benchmarks/performance.bench.out`
- `src/benchmarks/scenarios.bench.out`

8 Testing

FP-Deveric's validation story is built around Vitest. The default test configuration runs all `*.test.ts` and `*.spec.ts` files in a Node environment, while benchmark files `*.bench.ts` are configured separately and emit raw JSON to `src/benchmarks/bench-results.json`.

8.1 Test Topology

The current test tree covers five practical concerns:

- library behavior under `src/tests/libs/`
- core orchestration under `src/tests/core/`
- storage backends under `src/tests/storage/`
- integration and resilience under `src/tests/integration/`
- synthetic-data and benchmark helpers in `data-generator.test.ts` and `benchmarks/scenario-matrix.test.ts`

Shared deterministic fixtures live in `src/tests/fixtures/fingerprints.ts` and anchor most of the scorer and `DeviceManager` assertions.

8.2 Scoring and Generator Coverage

The current scorer tests are materially richer than the previous whitepaper described. `libs/confidence.test.ts` verifies:

- score bounds and coarse ordering for identical, near-identical, similar, and very different fixtures
- `tlshWeight = 0` and `tlshWeight = 1` behavior
- local weight overrides
- `useGlobalRegistry = false`
- every exposed breakdown dimension: `evidenceRichness`, `fieldAgreement`, `structuralStability`, `entropyContribution`, `attractorRisk`, `missingOneSide`, `missingBothSides`, and `computeAdaptiveStabilityWeights()`
- alignment between `calculateScoreBreakdown(...).composite` and `calculateConfidence(...)`

`data-generator.test.ts` focuses on scenario semantics rather than on claiming global determinism. It checks browser diversity, attractor-profile generation, blob-shaped canvas/WebGL/audio values, mutation behavior, and the expected label semantics of each scenario family.

`benchmarks/scenario-matrix.test.ts` acts as a regression gate on the canonical 17-scenario table. It asserts that, for the single-seed matrix, same-device cases remain at or above 85 and different-device cases remain below 85.

8.3 DeviceManager and Adapter Coverage

`core/device-manager.test.ts` is the largest behavioral suite in the project. It covers:

- new-device creation and returning-device reuse
- confidence and `matchConfidence` persistence
- dedup-cache hits, expiry, and explicit clearing
- adaptive weighting on stable histories
- post-processor enrichment and failure isolation

- logger and metrics callbacks
- `identifyMany()` ordering and shared-context behavior

`core/adapter-factory.test.ts` and `core/plugin-registrar.test.ts` cover the configuration and plugin lifecycle contracts exposed by the public API.

Storage coverage is split in two layers:

- `storage/adapter-contract.test.ts` defines the shared behavioral contract, but in the current repository it is exercised only against `InMemoryAdapter`.
- per-adapter suites validate backend-specific behavior for in-memory, SQLite, PostgreSQL, and Redis adapters.

The backend strategy is mixed by design:

- SQLite uses a real test database file.
- PostgreSQL uses mocked Drizzle primitives.
- Redis uses a hoisted in-memory mock of the `ioredis` API.

This keeps the default suite fast while still validating query-shape logic and duplicate-hash handling across the supported adapters.

Version 2.0.3 includes five new test modules under `src/tests/libs/`:

File	Tests	Coverage
<code>temporal-decay.test.ts</code>	17	<code>computeTemporalDecayFactor</code> , <code>decay integration</code> in <code>calculateScoreBreakdown</code> , <code>fresh-snapshot regression</code>
<code>drift.test.ts</code>	20	<code>computeDeviceDrift</code> pure function and <code>DeviceManager.analyzeDeviceDrift</code>
<code>identity-graph.test.ts</code>	25	<code>IdentityGraph</code> unit, <code>subnetKey</code> , <code>jaccardSimilarity</code> , <code>DeviceManager</code> integration (subnet + font edges)
<code>attractor-model.test.ts</code>	15	<code>DefaultAttractorModel</code> , <code>FrequencyTableAttractorModel</code> , <code>factory</code> , <code>integration</code> with <code>calculateScoreBreakdown</code>
<code>lsh-index.test.ts</code>	23	<code>createLshIndex</code> , <code>buildLshIndex</code> , LSH similarity behavior, <code>DeviceManager</code> integration

These bring the total test count to **302** assertions in the current tree.

8.4 Integration and Resilience Coverage

Two integration suites round out the test matrix:

- `integration/api-surface.test.ts` verifies the public barrel exports, adapter creation via public APIs, and a full identify/re-identify round trip.
- `integration/resilience.test.ts` forces adapter failures, TSLH edge cases, and empty-fingerprint inputs to ensure errors propagate or degrade safely.

This matters because FP-Devicer is middleware. It must behave predictably not only when the fingerprint is well-formed, but also when storage systems fail or the input is sparse.

8.5 Infrastructure Notes

The current validation setup has a few practical caveats that are worth stating explicitly:

- `npm test` passed locally during this re-evaluation with 20 test files passed and 1 skipped (296 tests passed, 6 skipped).
- Benchmarks are not part of `npm test`; they must be run separately with `npm run bench`.
- The benchmark suite currently fails in environments where `better-sqlite3` native bindings are not available, because `performance.bench.ts` creates a SQLite adapter at module load time.

8.6 Running the Suite

```
# Run tests
npm test

# Watch mode
npm run test:watch

# Coverage
npm run test:coverage

# Benchmarks
npm run bench
```

9 Usage Examples

9.1 Method 1: Simple (Using Defaults)

```
import { calculateConfidence } from "devicer.js";

const score = calculateConfidence(fpData1, fpData2);
// score is an integer in [0, 100]
```

9.2 Method 2: Advanced (Custom Weights & Comparators)

```
import { createConfidenceCalculator, registerPlugin } from "devicer.js";

// Contribute a globally-scoped plugin before creating any calculators
registerPlugin("userAgent", {
```

```

    weight: 25,
    comparator: (a, b) =>
        levenshteinSimilarity(
            String(a ?? "").toLowerCase(),
            String(b ?? "").toLowerCase(),
        ),
});

const calculator = createConfidenceCalculator({
    weights: {
        platform: 20,
        fonts: 20,
        screen: 15,
    },
    tlshWeight: 0.2,
});

const score = calculator.calculateConfidence(fpData1, fpData2);

```

9.3 Method 3: Enterprise (DeviceManager with Express)

```

import express from "express";
import { createInMemoryAdapter, DeviceManager } from "devicer.js";

const manager = new DeviceManager(createInMemoryAdapter(), {
    matchThreshold: 60,
    candidateMinScore: 30,
});

const app = express();
app.use(express.json());

app.post("/identify", async (req, res) => {
    const result = await manager.identify(req.body, {
        userId: (req as any).user?.id,
        ip: req.ip,
    });
    res.json(result);
    // → { deviceId, confidence, isNewDevice, linkedUserId }
});

app.listen(
    3000,
    () => console.log("FP-Devicer server ready at http://localhost:3000"),
);

```

9.4 Method 4: Deno / Oak (FP-Cicis Pattern)

```

import { Application, Router } from "oak";
import { DeviceManager } from "devicer";
import { createSqliteAdapter } from "../libs/sqlite.ts";

```

```

const adapter = createSqliteAdapter("./fp.db");
await adapter.init();

const manager = new DeviceManager(adapter, {
  matchThreshold: 60,
  candidateMinScore: 40,
});

const router = new Router();
router.post("/identify", async (ctx) => {
  const body = await ctx.request.body.json();
  const result = await manager.identify(body);
  ctx.response.body = result;
});

```

9.5 Method 5: Universal Plugins (ip-devicer + tls-devicer)

```

import express from "express";
import { createSqliteAdapter, DeviceManager } from "devicer.js";
import { createIpMiddleware, IpManager } from "ip-devicer";
import { createTlsMiddleware, TlsManager } from "tls-devicer";

// Construct the manager and companion plugins
const adapter = createSqliteAdapter("./fp.db");
const manager = new DeviceManager(adapter, { matchThreshold: 60 });
const ipManager = new IpManager({/* MaxMind DB options */});
const tlsManager = new TlsManager();

await adapter.init();

// Register companion plugins – order determines post-processor execution order
manager.use(tlsManager); // runs first: may boost result.confidence via JA4 match
manager.use(ipManager); // runs second: adds IP intelligence to the result

const app = express();
app.use(express.json());
app.use(createTlsMiddleware()); // attaches req.tlsProfile on every request

app.post("/identify", createIpMiddleware(), async (req: any, res) => {
  const result = await manager.identify(req.body, {
    userId: req.user?.id,
    ip: req.resolvedIp, // set by createIpMiddleware
    tlsProfile: req.tlsProfile, // set by createTlsMiddleware
  });

  console.log(result.deviceId); // stable device UUID
  console.log(result.confidence); // base score ± TLS confidence boost
  console.log(result.tlsConsistency); // { consistencyScore, ja4Match, factors,
  ... }
});

```

```
    console.log(result.ipEnrichment); // { country, asn, riskScore, isVpn, ... }
    console.log(result.enrichmentInfo); // { plugins: ["tls-devicer","ip-
devicer"], details, failures }

    res.json(result);
  });

app.listen(3000);
```

10 Installation & Integration

10.1 Installation

```
npm install devicer.js
```

10.2 FP-Snatch Integration

FP-Devicer is designed to pair with **FP-Snatch**, a companion open-source client-side JavaScript library that collects all fields of `FPUserDataSet` from a visitor's browser and POSTs them as JSON to a server endpoint.

```
<script src="./dist/bundle.js"></script>
<script defer>
  var agent = new window["snatch"]({
    url: "https://myserver.example/identify",
    method: "POST",
  });
  agent.send();
</script>
```

FP-Snatch collects, among other signals: user-agent, platform, screen dimensions, installed fonts, canvas fingerprint, WebGL fingerprint, audio fingerprint, plugin list, MIME types, hardware concurrency, device memory, timezone, language preferences, and UA-CH high-entropy values. It then sends the dataset as a JSON body matching the `FPUserDataSet` schema, ready for direct consumption by `DeviceManager.identify()` or `calculateConfidence()`.

A full working demonstration of the FP-Snatch → FP-Devicer pipeline is publicly hosted at cicis.info.

11 Conclusion & Future Work

FP-Devicer v2.0.3 delivers production-grade, open-source device intelligence with unmatched extensibility and accuracy for its class. Its hybrid structural-plus-TLSH scoring, adaptive per-device stability weighting, temporal signal decay, distribution-based drift detection, cross-device identity graphing, pluggable attractor risk models, and sub-

linear LSH candidate retrieval set it apart from simpler fingerprint hash libraries. Multi-backend persistence and zero-dependency observability interfaces ensure it integrates cleanly into any production stack.

The library is suitable for a wide range of use cases: account security and fraud detection, analytics de-duplication, personalisation without cookies, and research into browser fingerprinting techniques.

Community contributions are welcome. Possible future directions include:

- Official npm publishing and semantic versioning automation
- Additional storage adapters (DynamoDB, MongoDB, Cloudflare KV)
- Machine-learning-assisted comparator parameter tuning
- Browser extension support for richer signal collection
- Differential privacy mechanisms for privacy-preserving fingerprinting
- Periodic LSH index auto-rebuild triggered by adapter snapshot count growth
- Persistence for IdentityGraph state across process restarts

12 License

See `license.txt` in the repository root. FP-Deviser is released under a permissive open-source license.

13 Acknowledgements

Special thanks to Scott VanRavenswaay for advisory on certain technical challenges and architectural decisions, and to the open-source community (particularly GitHub) for their pull requests and issues.